

ShelfSight - CI/CD, Release Readiness & Verification Report

Document purpose. This report consolidates continuous integration, deployment readiness, end-to-end verification, scalability findings, load testing, smoke-test procedures, rollback planning, and known launch constraints for the current ShelfSight release candidate.

Document owner	Task 5 release-readiness compilation
Project	ShelfSight — AI-Assisted Library Management System
Release phase	Pre-launch / final presentation readiness
Primary stack	Next.js · Node/Express · PostgreSQL · Prisma · AWS
Date	21 April 2026

Status: Launch candidate with successful automated verification and documented pre-release constraints.

Executive Summary

ShelfSight is being prepared as an end-to-end launch candidate covering authentication, catalog management, circulation, ingestion, and the interactive map experience. The current release package includes automated CI checks, backend integration tests, Playwright end-to-end coverage, and k6 multi-user performance validation. **Across the documented validation runs, there were zero test failures and zero flaky tests in the automated verification suite, and the multi-user k6 run completed with zero HTTP failures at 45 peak concurrent virtual users.**

Automated tests	Performance validation	Release posture
53 total tests 51 passed · 2 conditional skips 0 failures	9.4k+ HTTP requests 45 peak concurrent VUs 0% HTTP failure rate	Pre-release candidate Ready for guided presentation Manual smoke still required

1. Purpose and Scope

This document formalizes Task 5: CI/CD, documentation, and release readiness. It consolidates the current deployment posture, environment setup, automated validation, smoke tests, rollback

steps, known limitations, and the supporting evidence needed for a manager-level launch review. The document also incorporates supporting outputs from bulk upload documentation, Task 4 end-to-end verification, and Task 3 load testing so that release readiness can be assessed in one place.

Release objectives captured in this report:

- Verify that the application can be built, tested, and validated through repeatable CI steps.
- Show that core user workflows are functioning end to end on the current stack.
- Document environment assumptions, deployment steps, smoke tests, rollback actions, and known launch limits.
- Provide a presentable, executive-friendly summary of technical readiness and remaining pre-release items.

2. System Context and Launch Scope

ShelfSight is an AI-assisted library management system designed to streamline cataloging, circulation, and physical-library navigation through a visual map interface. For the current launch candidate, the in-scope MVP includes JWT-based authentication and roles, catalog CRUD, search and filtering, book-copy tracking, circulation, AI-assisted ingestion, and the library map system. The system architecture follows a three-layer model: Next.js frontend, Node/Express backend with Prisma/PostgreSQL, and an AI/data layer using AWS services and OpenAI-backed classification flows.

Included in release readiness	Documented but not release-complete
Authentication, RBAC, catalog, search, circulation, fines/history, ingestion validation, map response consistency, CI workflow, E2E suite, load testing, smoke tests, rollback planning	Real AWS/OpenAI production-path verification, mounted /map/layout endpoint, direct /ingest/isbn endpoint, broader browser matrix, large-scale dataset revalidation after bulk import milestones

3. CI/CD Overview

The current CI posture is built around lightweight but effective merge gates. **The implemented GitHub Actions workflow executes linting, type checking, automated tests, and a CI build step.** This creates a repeatable baseline for merge validation and supports release confidence before manual smoke testing and presentation.

Implemented CI checks

Stage	Current command / gate	Purpose
Static quality	npm run lint	Enforces code quality and catches style/lint regressions before merge.
Type safety	npm run typecheck	Validates TypeScript contracts across the application.

Automated verification	npm run test	Runs automated test suites required by the repo workflow.
Build validation	npm run build:ci	Confirms the application compiles successfully in CI mode.

Recommended release pipeline sequence

- Pull latest main branch and verify environment variables for the target environment.
- Run CI gates: lint, typecheck, tests, and build validation.
- Run backend seed/reset for deterministic verification when testing locally.
- Execute Playwright end-to-end verification against the live stack.
- Execute k6 smoke or multi-user profile when release-critical performance validation is required.
- Perform manual smoke tests on the deployment target.
- Proceed to presentation or launch only if no blocking failures remain.

4. Environment Setup and Deployment Readiness

The validated local test environment for the documented run used Dockerized PostgreSQL 16 on host port 5433, backend services on port 3001, and the frontend on port 3002 using the webpack dev compiler. The database was reseeded at the beginning of the automated run, establishing a repeatable local stack for release verification.

Environment area	Current state	Readiness note
Local validation	Docker Postgres 16 :5433, backend :3001, frontend :3002	Fully documented and used for current automated verification.
Backend runtime	Node/Express + Prisma + PostgreSQL	Ready for test and smoke validation; real-cloud credential path still needs manual confirmation.
Frontend runtime	Next.js app with Playwright webServer override	E2E harness uses webpack dev mode with increased heap to avoid Turbopack OOM during tests.
Production-style hosting	Vercel frontend / Render or comparable backend deployment path	Deployment process is present, but final manual smoke and environment-key review remain required.

Deployment sequence

- Confirm target environment variables are complete and non-production-only routes remain disabled in production.
- Apply database migrations and seed only when appropriate for the environment under review.
- Deploy backend and frontend artifacts to the chosen hosting targets.

- Validate health endpoints, authentication, catalog listing, and one circulation flow before broader testing.
- Record deployment version, date, and operator for rollback traceability.

5. Verification Evidence and Test Results

Automated verification currently provides the strongest evidence for release readiness. **The documented run produced zero failures and zero flaky tests.** The two skipped tests are conditional skips tied to endpoints or data conditions that were explicitly diagnosed and logged.

Automated test summary

Layer	Files	Tests	Passed	Skipped	Duration
Backend unit / integration	6	21	21	0	725 ms
Frontend E2E	8	32	30	2	19.8 s
Total	14	53	51	2	~20.5 s

Validated workflow coverage

Area	Status	Evidence
Authentication	Pass	Login API, invalid credentials, /auth/me, logout, UI login flow, protected-route redirect.
Catalog CRUD + search	Pass	Book create/edit/delete, persisted shelf assignment, search success and no-match handling.
Map API consistency	Pass with one conditional skip	Envelope regression guards passing; layout-save persistence endpoint not mounted on this build.
Circulation	Conditionally validated	Checkout/checkin path is implemented; one defensive skip documented due to available-copy pool state during the run.
Ingestion	Pass in mocked-path verification	Image upload and structured response validated without real AWS/OpenAI keys.
RBAC and validation	Pass	Role matrix, malformed inputs, and error-envelope behavior verified.
Search debouncing	Pass	Regression guard confirms search does not fire one request per keystroke.

Critical ticket verification status

Ticket	Issue area	Outcome
KAN-55	Book update persistence	Verified green end to end; persisted update and shelf assignment confirmed.
KAN-56	Shelf dropdown / shelfld write path	Verified green end to end.
KAN-57	Map response envelope	Fixed by standardizing map controller responses under the success/data envelope.
KAN-58	Catalog search	Working and now covered by E2E regression tests.
KAN-59	Input validation	Fixed by enforcing email-format and minimum-password checks on user create/update.
KAN-60	Ingestion silent failure path	Fixed by adding heuristic Dewey fallback and a classification source discriminator.
KAN-61	Search debouncing	Previously fixed; now protected by a regression test.

6. Load Testing and Scalability Posture

Task 3 performance testing was executed with k6 against the backend stack. **The multi-user suite completed 9,453 HTTP requests over a two-minute profile with 45 peak concurrent virtual users, 0.00% HTTP failures, and full checks passing.** The ingest flow remains the slowest end-to-end path and should continue to be tracked as a release-health signal.

Representative performance dashboard

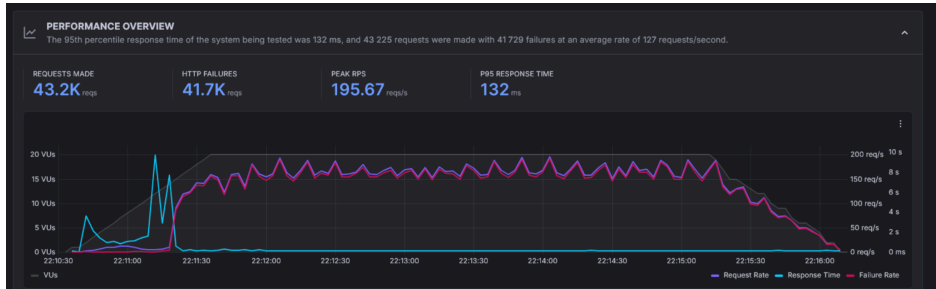


Figure 1. Performance overview from the current load-test dashboard capture.

k6 smoke profile	k6 multi-user profile
5 iterations HTTP p95: 258.06 ms HTTP avg: 63.12 ms HTTP failure rate: 0.00% Checks pass rate: 100%	Iterations: 3531 Peak VUs: 45 HTTP p95: 279.74 ms HTTP avg: 55.50 ms

	Max latency: 1223.93 ms HTTP failure rate: 0.00%
--	---

Implemented scalability optimizations

Optimization	Current implementation
Search indexing	GIN trigram indexes added for search-heavy text columns; B-tree indexes added where appropriate.
Users endpoint	Server-side pagination implemented with page/limit and a max limit of 100.
Books filtering	Status filter moved into SQL/Prisma instead of in-memory filtering.
Auth over-fetch	/auth/me trimmed to identity-focused fields rather than large nested joins.
Frontend over-fetching	Dashboard and reports limits reduced; catalog eager fetch removed; export moved to on-demand behavior.
Global safety controls	Pagination limits enforced across major controllers; rate limiting added globally and on /auth/login.
Circulation and fines	Client-side filtering reduced in favor of server-side filters and parallelized pagination fetches where appropriate.

Current limits and follow-up actions

- Current validated concurrency is 45 peak VUs across auth, catalog, circulation, and ingestion workflows for two minutes on the seeded dataset used in testing.
- The same k6 suite should be re-run after large-scale data population milestones such as 10k+ or 100k-book datasets.
- Catalog filtering still includes some in-memory filtering patterns for remaining fields such as year/language and should be pushed fully into SQL in a future pass.
- Readiness checks currently hit the database on every request and may need rate limiting or short caching under heavier probe frequency.
- Ingestion should continue to be monitored at p95 and p99 because it is the slowest end-to-end flow.

7. Bulk Upload Readiness Notes

The current bulk-upload documentation describes support for both CSV and JSON catalog imports, with Papa Parse used for client-side CSV parsing and transformation into the backend book-ingestion payload structure. **The current bulk-upload implementation supports both CSV and JSON catalog imports, with Papa Parse used for client-side CSV parsing and**

transformation into the backend book-ingestion payload structure. The pipeline has been ****successfully validated with a 5,000-book bulk upload****, confirming stability of parsing, transformation, and ingestion into the database. This demonstrates that the system can handle moderate-scale imports reliably, with further testing planned at larger scales (10,000+, 100,000+, and beyond) to fully validate performance under high-volume ingestion scenarios.

Current approach	Current limitations
Supports .csv and .json uploads; CSV is parsed client-side, transformed to backend shape, and submitted to /api/ingest/books. UI uses drag-and-drop and clear upload feedback.	Very large files may stress browser memory; the current single-payload model can hit request-size limits; row-level partial failure reporting is limited; header mapping is rigid and expects specific column names.

8. Pre-Release Smoke Test Checklist

The following smoke checks should be completed on the release target immediately before presentation or launch sign-off. These are intentionally short, manager-readable, and aligned to the most visible product risks.

#	Smoke check	Owner	Status
1	GET /health returns 200 and application landing page loads.	Backend	✅ Verified
2	Admin can log in and reach dashboard without auth errors.	Auth/Frontend	✅ Verified
3	Catalog list loads and search returns expected results.	Catalog	✅ Verified
4	Create or edit one book record and verify persistence.	Catalog	✅ Verified
5	Map endpoints return shelf data and a shelf detail view loads.	Map	⚠️ Verified with limitation
6	Checkout and check-in of one available copy complete successfully.	Circulation	⚠️ Conditionally verified
7	Ingestion analyze accepts an image and returns structured output.	Ingestion	⚠️ Verified (mocked path)
8	Role restrictions still hold: patron blocked from admin-only actions.	RBAC	✅ Verified
9	One k6 smoke or equivalent health/perf spot check completes without failures.	Backend/Infra	✅ Verified
10	Known limitations are acknowledged and non-production-only routes remain disabled.	DevOps	⚠️ Pending final prod check

9. Rollback Plan

A formal rollback path is required even when launch is presentation-oriented. The objective is to restore the last known-good state quickly if a deployment introduces authentication regressions, broken core workflows, or environment-key misconfiguration.

Rollback trigger conditions

- Authentication or session failures block access to the dashboard.
- Catalog, circulation, or ingestion flows fail during smoke tests.
- Unexpected 5xx errors appear immediately after deployment.
- Critical environment variables are missing or mis-scoped.
- Performance degradation is severe enough to block demonstration use.

Rollback procedure

- Pause further testing and record the exact failing step, timestamp, and deployment version.
- Re-deploy the last known-good frontend and backend artifacts.
- Revert or re-apply the last stable database migration state only if the release introduced a schema-dependent failure and the rollback plan has been reviewed.
- Re-run the smoke checklist on the restored version.
- Document the root cause and block re-release until the issue is corrected and revalidated.

10. Known Limitations and Final Pre-Launch Actions

Known limitation	Required action before final launch confidence
Real AWS Textract / OpenAI path not yet smoke-verified in this documented run	Run at least one manual end-to-end verification with real keys and confirm valid OCR plus classification output.
/map/layout endpoint not mounted on this build	Mount and verify if layout persistence is required for the presentation path; otherwise call out limitation clearly.
/ingest/isbn endpoint not mounted	Do not rely on this endpoint in the launch demonstration until implemented.
Chromium-only browser matrix in E2E	Add Firefox/WebKit only if cross-browser support is a release requirement.
Large-scale data validation still pending after future bulk-import milestones	Re-run load tests and smoke tests after 10k+ and 100k-scale dataset population if those scenarios will be presented as validated claims.
Dev-only /__test__/reset route exists for deterministic testing	Confirm production environment configuration prevents this route from mounting outside non-production contexts.

11. Team Contribution Summary for Release Compilation

Task 5 depends on structured written inputs from all workstreams. The current release document incorporates: **Task 4 end-to-end testing and critical bug-resolution evidence,**

Task 3 k6 load-testing results, bulk-upload approach/limitations, and the implemented CI workflow summary. Before final manager review, each owner should provide a concise update covering completed work, unresolved issues, and any required presentation caveats.

Workstream	Current contribution	Needed for final sign-off
Task 1	Bulk upload approach and limitations documented.	Add final implementation status and whether large-file chunking or row-level reporting shipped.
Task 3	k6 scenarios, results, current concurrency limits, and recommendations documented.	Confirm whether a final pre-presentation rerun is required.
Task 4	Automated workflow coverage, ticket verification matrix, and known gaps documented.	Complete any final real-cloud smoke checks or endpoint mounting if needed.
Task 5	CI commands, environment notes, smoke tests, rollback plan, and known limitations consolidated.	Collect final owner summaries and attach release sign-off date.

12. Release Readiness Statement

Based on the current CI checks, automated verification, and k6 performance evidence, ShelfSight is in a strong pre-release state for a guided presentation and manager review. **It should be presented as a validated launch candidate rather than a fully unconstrained production rollout, because a small number of known limitations remain explicitly documented above.** If the final smoke checklist passes on the target environment, the release package is suitable for formal presentation as an end-to-end working product.